



Universidad Tecnológica Nacional
Facultad Regional Córdoba

Trabajo práctico N2: programación de la placa BluePill en BareMetal

Ejercicio Práctico N1: interrupciones externas y el NVIC

Técnicas Digitales II

4R2

Gastón Grasso 401892

Franco Palombo 401910

Angelo Prieto 401012

Última actualización: 8 de junio de 2026

Índice

1	Introducción	2
2	Desarrollo	3
2.1	Ejercicio Práctico N1: interrupciones externas y el NVIC	3
2.1.1	Archivo de Inicialización y Tabla de Vectores	3
2.1.2	Script de Enlazado	4
2.1.3	Código Fuente Principal en C	6
2.1.4	Relación y Flujo de Ejecución entre los Componentes	9
3	Conclusiones	10
4	Fuentes	11

1 Introducción

Este informe técnico documenta el desarrollo y la resolución del Ejercicio Práctico N.º 1 del Trabajo Práctico Nro 2 de la materia Técnicas Digitales II. Las actividades propuestas se llevan a cabo sobre la placa de desarrollo Blue Pill, la cual incorpora un microcontrolador STM32F103C8T6 basado en el núcleo ARM Cortex-M3. El enfoque de programación implementado es Bare Metal puro, interactuando de forma directa con los registros de los periféricos a través de punteros de memoria.

La guía de trabajos prácticos plantea una serie de cinco ejercicios orientados a comprender el funcionamiento y configuración de diversos módulos del microcontrolador:

1. Ejercicio Práctico N.º 1 (Interrupciones Externas y el NVIC): Configuración del controlador de interrupciones vectorizadas y anidadas (NVIC) para gestionar eventos asíncronos provenientes de un pin de entrada (botón) mediante interrupción externa (EXTI), modificando el estado de un LED.
2. Ejercicio Práctico N.º 2 (Conversión Analógica-Digital): Configuración del ADC para la lectura del sensor de temperatura interno y de un canal analógico externo.
3. Ejercicio Práctico N.º 3 (Migración a CMSIS y Gestión de Tiempos): Incorporación del estándar CMSIS para mejorar la legibilidad y uso del temporizador del sistema (SysTick) para generar retardos precisos.
4. Ejercicio Práctico N.º 4 (Comunicación Serie UART): Establecimiento de comunicación bidireccional con una PC mediante el periférico USART para la transmisión de estados y recepción de comandos.
5. Ejercicio Práctico N.º 5 (Control de Potencia con Timers): Generación de señales de modulación por ancho de pulso (PWM) empleando los temporizadores de propósito general para variar la intensidad lumínica.

Cabe destacar que el presente documento aborda de manera exclusiva la resolución del Ejercicio Práctico N.º 1. Los ejercicios subsiguientes (del 2 al 5) se desarrollarán de forma progresiva en futuros informes. El objetivo principal de esta primera etapa consiste en comprender la tabla de vectores de interrupción, el funcionamiento de la línea EXTI y la habilitación de interrupciones en el NVIC sin utilizar bibliotecas de alto nivel.

2 Desarrollo

En este capítulo se detalla la resolución técnica del primer ejercicio práctico, correspondiente al diseño e implementación del sistema de gestión de interrupciones externas utilizando la placa BluePill en un entorno Bare Metal.

2.1 Ejercicio Práctico N1: interrupciones externas y el NVIC

El objetivo de este ejercicio consiste en configurar el microcontrolador STM32F103C8T6 para que el LED de depuración interno (conectado al pin PC13) permanezca parpadeando periódicamente en el bucle principal, mientras que un pulsador (conectado al pin PC15) modifica de forma asíncrona, mediante interrupción externa, el estado de un LED externo (conectado al pin PC14).

Para llevar a cabo la resolución de este ejercicio práctico, el proyecto se divide en tres archivos fundamentales: el código de arranque en ensamblador (`crt.s`), el script de enlazado (`linker.ld`) y el código fuente principal en C (`main.c`). A continuación, se detalla el análisis de cada uno de ellos y la relación funcional que existe entre sí.

2.1.1 Archivo de Inicialización y Tabla de Vectores

El archivo `crt.s`, cuyo código se expone en el listado 2.1, representa el código de inicio (startup) del sistema. Al energizar el microcontrolador, la CPU del Cortex-M3 inicializa el puntero de pila (Stack Pointer) con el valor ubicado en la dirección de memoria `0x08000000` (inicio de la memoria FLASH) y salta a la dirección del vector de reinicio (Reset Vector) ubicada en `0x08000004`.

En este archivo se define la sección `.isr_vector`, la cual contiene la tabla de vectores de interrupción requerida por la arquitectura ARM Cortex-M3 [Cor]. Las primeras posiciones corresponden a excepciones internas del núcleo (Reset, NMI y Hard Fault). Las posiciones del núcleo restantes (de la 4 a la 15) se rellenan con ceros mediante la directiva `.rept 12`, ya que no son utilizadas en esta aplicación.

A continuación de las excepciones del núcleo, se posicionan los vectores de interrupción específicos de los periféricos de la placa STM32. La interrupción asociada a las líneas externas EXTI15 a EXTI0 corresponde a la posición física 40 de la tabla de periféricos (posición 56 en la tabla global, considerando los 16 vectores del núcleo). Por lo tanto, se agregan 40 palabras en cero utilizando `.rept 40` para ocupar los espacios de los periféricos anteriores y, finalmente, se registra la función `EXTI15_10_IRQHandler` en la posición 56.

El punto de entrada físico del firmware es el manejador de reset `_reset`, el cual llama a la función principal en C mediante la instrucción de ramificación con enlace `bl main`. Si la función principal retornara, el microcontrolador quedaría en un bucle infinito para evitar comportamientos erráticos.

```

1 .syntax unified
2 .cpu cortex-m3
3 .thumb
4 .extern _esstack
5
6 .section .text.manejadores
7
8 // Función por defecto si ocurre una interrupción no implementada
9 .thumb_func
10 .weak Default_Handler
11 Default_Handler:
12     b .
13
14 // Definición de vectores como débiles (weak)
15 .weak NMI_Handler
16 .thumb_set NMI_Handler, Default_Handler
17 .weak HardFault_Handler
18 .thumb_set HardFault_Handler, Default_Handler
19 .weak EXTI15_10_IRQHandler
20 .thumb_set EXTI15_10_IRQHandler, Default_Handler
21
22 // Tabla de vectores de interrupción
23 .section .isr_vector,"a",%progbits
24 .word _esstack           // Posición 0: Puntero de pila
25 .word _reset            // Posición 1: Vector de Reset
26 .word NMI_Handler       // Posición 2: NMI
27 .word HardFault_Handler // Posición 3: Falla de hardware
28
29 // Rellena las posiciones de excepciones del núcleo restantes (de la 4 a la 15)
30 .rept 12
31 .word 0
32 .endr
33
34 // Rellena las primeras interrupciones de periféricos del STM32 (de la 16 a la 55)
35 // Esto cubre los vectores desde WWDG hasta la interrupción número 39
36 .rept 40
37 .word 0
38 .endr
39 .word EXTI15_10_IRQHandler // Posición 56, dirección física exacta para la línea EXTI15_10
40
41 // Sección de código de Reset
42 .section .text.reset
43 .thumb_func
44 .global _reset
45 _reset:
46     bl main
47     b .

```

Listing 2.1: Archivo de inicialización y tabla de vectores (crt.s).

2.1.2 Script de Enlazado

El archivo `linker.ld`, presentado en el listado 2.2, es el script de configuración del enlazador (GNU Linker). Su función principal es indicar cómo deben distribuirse las secciones de código y datos dentro del espacio de direcciones de memoria del microcontrolador.

En primer lugar, define el bloque `MEMORY`, especificando que el chip `STM32F103C8T6` posee una memoria `FLASH` de lectura y ejecución (`rx`) de 64 kB con origen en `0x08000000`, y una memoria `RAM` de lectura, escritura y ejecución (`xrw`) de 20 kB con origen en `0x20000000`.

El símbolo `_esstack` se calcula sumando el origen de la `RAM` con su longitud, apuntando así a la última posición de la memoria `RAM` (pila invertida o descendente). Este símbolo es exportado y consumido directamente en la primera posición de la tabla de vectores en `crt.s`.

Dentro del bloque `SECTIONS`, la directiva `KEEP(*(.(isr_vector))` asegura que la sección de la tabla de vectores de interrupción no sea eliminada por optimizaciones del compilador y que se ubique de manera obligatoria al principio de la memoria FLASH. Posteriormente, se disponen las secciones de código y constantes (`.text` y `.rodata`) en la FLASH, mientras que las secciones de variables globales (`.data` y `.bss`) se mapean para su uso en la RAM.

```
1 MEMORY
2 {
3     FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 64K
4     RAM (xrw)  : ORIGIN = 0x20000000, LENGTH = 20K
5 }
6
7 /* Punto de entrada al arrancar */
8 ENTRY(_reset)
9
10 /* Dirección final de la RAM para inicializar la pila */
11 _esstack = ORIGIN(RAM) + LENGTH(RAM);
12
13 SECTIONS
14 {
15     /* Asegura la colocación de la tabla de vectores al inicio de la FLASH */
16     .isr_vector :
17     {
18         KEEP(*(.(isr_vector))
19     } > FLASH
20
21     /* Código del programa y constantes */
22     .text :
23     {
24         *(.text)
25         *(.text*)
26         *(.rodata)
27         *(.rodata*)
28     } > FLASH
29
30     /* Direcciones de control para variables globales (en caso de usarse en el futuro) */
31     _sdata = LOADADDR(.data);
32
33     .data :
34     {
35         _sdata = .;
36         *(.data)
37         *(.data*)
38         _edata = .;
39     } > RAM AT> FLASH
40
41     .bss :
42     {
43         _sbss = .;
44         *(.bss)
45         *(.bss*)
46         *(COMMON)
47         _ebss = .;
48     } > RAM
49 }
```

Listing 2.2: Script de enlazado de memoria (`linker.ld`).

2.1.3 Código Fuente Principal en C

El archivo `main.c`, expuesto en el listado 2.3, implementa la inicialización de los registros de control y la rutina de servicio para procesar las interrupciones externas. Al no contar con los archivos de cabecera estándar del fabricante, se definen de manera explícita las direcciones físicas de los periféricos basándose en el manual de referencia [Stm].

Mapeo de Registros

- **GPIOC:** Con dirección base en `0x40011000`, define los registros `GPIOC_CRL` y `GPIOC_CRH` para configurar el modo de funcionamiento de los pines, `GPIOC_ODR` para manejar la salida de datos o las resistencias de pull-up/pull-down internas, y `GPIOC_BSRR` para modificar estados de pines de forma atómica.
- **RCC:** Dirección base `0x40021000`. El registro `RCC_APB2ENR` permite habilitar el reloj de los periféricos conectados al bus APB2 (en este caso, el puerto C y el módulo de funciones alternas AFIO).
- **AFIO:** Dirección base `0x40010000`. El registro `AFIO_EXTICR4` configura qué puerto físico se asocia a las líneas de interrupción externa de la 12 a la 15.
- **EXTI:** Dirección base `0x40010400`. Contiene registros como `EXTI_IMR` (máscara de interrupciones), `EXTI_FTSR` (detección por flanco de bajada) y `EXTI_PR` (registro de petición pendiente).
- **NVIC:** Se accede al registro de habilitación del núcleo ARM Cortex-M3 en la dirección `0xE000E104` (`NVIC_ISER1`) para controlar el estado del canal `EXTI15_10`.

Configuración Paso a Paso en la Función Principal

Dentro del ciclo de inicialización en la función `main()`, se ejecutan las siguientes acciones en secuencia:

1. **Habilitación del reloj de periféricos:** Se configuran los bits 4 y 0 de `RCC_APB2ENR` para suministrar señal de reloj al puerto GPIOC y al periférico AFIO. Sin esta habilitación, cualquier escritura en los registros de dichos módulos será ignorada por el hardware.
2. **Configuración de PC13 y PC14 como salidas:** Se limpian los campos de configuración en `GPIOC_CRH` para los pines 13 y 14, y se establece en ambos el valor `0x3` (modo salida de propósito general push-pull con una velocidad máxima de conmutación de 50 MHz). En el puerto PC13 se encuentra conectado el led interno y en el puerto PC14 se conectará un led externo que será controlado mediante la interrupción externa.
3. **Configuración de PC15 como entrada con Pull-up:** Se configura el campo de control en `GPIOC_CRH` con el valor `0x8` (modo entrada con resistencia interna) y luego se escribe un '1' en el bit 15 del registro `GPIOC_ODR` para configurar dicha resistencia como pull-up. Esto mantiene el pin en un nivel lógico alto por defecto, cayendo a nivel bajo al pulsar el botón conectado a tierra.

4. Asociación de la interrupción al Pin PC15: Se escribe el valor 0x2 en el registro AFIO_EXTICR4 en el rango correspondiente a la línea 15. Esto le indica al multiplexor de interrupciones del microcontrolador que la línea EXTI15 debe responder al Puerto C (donde se encuentra el pulsador).
5. Configuración de disparo de la interrupción externa: Se escribe un '1' en el bit 15 de EXTI_FTSR para detectar flancos descendentes. De esta manera, el evento de interrupción se disparará en el instante exacto en el que el botón sea presionado.
6. Desenmascaramiento de la línea EXTI15: Se habilita la propagación de la señal escribiendo un '1' en el bit 15 del registro de máscara EXTI_IMR.
7. Activación en el NVIC: Se escribe un '1' en el bit 8 de NVIC_ISER1. Como el canal EXTI15_10 posee el número de interrupción (IRQ) 40, este se activa mediante el bit 8 de la segunda palabra del registro de habilitación del NVIC ($40 - 32 = 8$).

Una vez completada la inicialización, la CPU ejecuta de forma ininterrumpida el bucle principal `while(1)`, que alterna el estado del LED interno en PC13 mediante retardos por software (bucles vacíos de larga duración).

Rutina de Servicio de Interrupción (ISR)

Cuando se produce un flanco descendente en el pin PC15, la CPU suspende la ejecución en el programa principal, almacena los registros de estado en la pila e ingresa a la ISR `EXTI15_10_IRQHandler`. Dentro de esta rutina:

1. Se comprueba si la interrupción proviene de la línea 15 mediante la verificación del bit correspondiente en la máscara de interrupción.
2. Se invierte el estado del LED externo en PC14 aplicando una operación XOR a nivel de bit sobre el registro `GPIOC_ODR`.
3. Limpieza del flag de interrupción: Se escribe un '1' en el bit 15 del registro de estado `EXTI_PR`. En el controlador de interrupciones externas de la arquitectura STM32, la limpieza de las banderas pendientes se realiza por software escribiendo un '1' en la posición correspondiente. Omitir este paso provocaría que, al salir de la ISR, el núcleo detecte la interrupción como activa de forma permanente, volviendo a ejecutar la ISR de manera indefinida y bloqueando el sistema.

```

1 #include <stdint.h>
2
3 typedef volatile uint32_t register_t;
4
5 #define GPIOC_BASE_ADDRESS 0x40011000
6 #define GPIOC_CRL      (*(register_t *) (GPIOC_BASE_ADDRESS))           // GPIOC cfg register L (0-7)
7 #define GPIOC_CRH      (*(register_t *) (GPIOC_BASE_ADDRESS + 0x04U))   // GPIOC cfg register H (8-15)
8 #define GPIOC_ODR      (*(register_t *) (GPIOC_BASE_ADDRESS + 0x0CU))   // GPIOC Output data reg
9 #define GPIOC_BSRR      (*(register_t *) (GPIOC_BASE_ADDRESS + 0x10U))  // GPIOC Set/Reset reg (atomic)
10 #define GPIOC_BRR       (*(register_t *) (GPIOC_BASE_ADDRESS + 0x14U))  // GPIOC Reset reg (atomic)
11
12 #define RCC_BASE_ADDRESS 0x40021000

```



```

13 #define RCC_APB2ENR (*(register_t *) (RCC_BASE_ADDRESS + 0x18U)) // Advanced Peripheral Bus 2 clk
    en reg
14
15 // Periférico AFIO (Alternate Function I/O)
16 #define AFIO_BASE_ADDRESS 0x40010000
17 // Registro de configuración de interrupciones externas 4 (maneja pines 12 a 15)
18 #define AFIO_EXTICR4 (*(register_t *) (AFIO_BASE_ADDRESS + 0x14U))
19
20 // Periférico EXTI (External Interrupt Controller)
21 #define EXTI_BASE_ADDRESS 0x40010400
22 #define EXTI_IMR (*(register_t *) (EXTI_BASE_ADDRESS + 0x00U)) // Interrupt Mask Register
23 #define EXTI_FTSR (*(register_t *) (EXTI_BASE_ADDRESS + 0x0CU)) // Falling Trigger Selection
    Register
24 #define EXTI_PR (*(register_t *) (EXTI_BASE_ADDRESS + 0x14U)) // Pending Register
25
26 // Periférico NVIC
27 // El NVIC está dentro del propio núcleo ARM (Cortex-M3 Internal Core Peripheral)
28 #define NVIC_ISER1 (*(register_t *) (0xE000E104U)) // Interrupt Set-Enable Register 1
29
30 int main() {
31     // encendido de los clocks
32     RCC_APB2ENR |= (1 << 4) | (1 << 0); // enciende clock de la GPIO y de la AFIO
33
34     // configuración de la GPIOC 13 como salida (led interno)
35     GPIOC_CRH &= (~((register_t) 0xF << 20)); // Mascara para escribir los bits de config del
        GPIOC_13
36     GPIOC_CRH |= 0x3 << 20; // Mode: 0b11: output mode 50MHz. Config: 0b00: GPO P-P
37
38     // configuración de la GPIOC 14 como salida (led externo)
39     GPIOC_CRH &= (~((register_t) 0xF << 24)); // Mascara para escribir los bits de config del
        GPIOC_14
40     GPIOC_CRH |= 0x3 << 24; // Mode: 0b11: output mode 50MHz. Config: 0b00: GPO P-P
41
42     // configuración de la GPIOC 15 como entrada pull up
43     GPIOC_CRH &= (~((register_t) 0xF << 28)); //Puerto GPIOC_15
44     GPIOC_CRH |= 0x8 << 28; //Mode: input pullup/pulldown
45     GPIOC_ODR |= (1 << 15); //define GPIO_C15 explícitamente la entrada como pullup
46
47     // configuración de la interrupción
48     AFIO_EXTICR4 |= 0x2 << 12; // indica que puerto C pin 15 será interrupción
49     EXTI_FTSR |= 0x1 << 15; // establece que el flanco de bajada activará la IQR
50     EXTI_IMR |= 0x1 << 15; // desenmascara la interrupción en el pin correspondiente
51     NVIC_ISER1 |= 0x1 << 8; // habilita interrupción en el NVIC
52
53     while (1) {
54         GPIOC_BSRR = 1 << 13; // offset para la parte de SET del registro
55         for (int i = 0; i < 500000; i++); // delay arbitrario
56         GPIOC_BSRR = 1 << (13 + 16); // +16 es el offset para la parte de RESET del registro
57         for (int i = 0; i < 500000; i++); // delay arbitrario
58     }
59
60     return 0;
61 }
62
63 void EXTI15_10_IRQHandler(void) {
64     // verifica si la interrupción fue realmente causada por el pin 15
65     if (EXTI_IMR & (1 << 15)) {
66         GPIOC_ODR ^= 1 << 14; // toggle led externo
67
68         EXTI_PR = (1 << 15); //limpia flag de interrupción, si no se hace esto
69                             //al salir de la función el micro volverá a entrar
70                             //inmediatamente
71     }
72 }

```

Listing 2.3: Código fuente principal e implementación de la ISR (main.c).

2.1.4 Relación y Flujo de Ejecución entre los Componentes

La interacción de los tres archivos de desarrollo analizados conforma la base necesaria para la correcta ejecución en Bare Metal:

1. El archivo de enlazado (`linker.ld`) asigna de manera rígida los límites de la memoria física y establece que la tabla de vectores definida en `crt.s` comience exactamente en la posición `0x08000000`.
2. El código en ensamblador (`crt.s`) posiciona la dirección física de la rutina `EXTI15_10_IRQHandler` en el vector número 40 del NVIC (posición 56 de la tabla).
3. Durante la compilación y el enlace, el compilador traduce la función definida en `main.c` en una dirección física de memoria, la cual el enlazador copia en el vector asignado dentro de la tabla de interrupciones.
4. Cuando la interrupción física se produce en el microcontrolador, el NVIC consulta la dirección de memoria asociada al vector 40 en la FLASH y salta hacia la función implementada en C, logrando la interacción asíncrona.

3 Conclusiones

Tras el análisis e implementación del Ejercicio Práctico N.º 1, se concluye que se han alcanzado satisfactoriamente los objetivos propuestos. La utilización de una interrupción externa gestionada a través del NVIC y el módulo EXTI ha permitido resolver de manera eficiente la detección y el procesamiento de eventos asíncronos (la pulsación del botón en PC15), rompiendo el flujo secuencial del programa principal sin requerir un muestreo periódico (polling).

El desarrollo en un entorno Bare Metal puro (sin sistemas operativos ni capas de abstracción complejas) ha resultado fundamental para comprender de manera profunda el comportamiento a bajo nivel de la arquitectura ARM Cortex-M3. La configuración manual de registros específicos como los de habilitación de reloj en el RCC, control de puertos GPIO, asociación de líneas en el AFIO, configuración de flancos en el EXTI, y activación en el NVIC proporciona un control absoluto sobre el hardware y optimiza el uso de recursos de memoria y CPU.

Asimismo, la implementación de la tabla de vectores en el archivo de inicio en ensamblador y la definición exacta de la distribución física en el script de enlazado ponen de manifiesto la estrecha relación que existe entre el software desarrollado y la estructura del hardware subyacente. Se constata la vital importancia de limpiar el flag pendiente de interrupción en el registro correspondiente antes de finalizar la rutina de servicio, ya que la omisión de este paso invalida el comportamiento esperado del microcontrolador al generar bucles infinitos de atención.

4 Fuentes

- [Cor] *Cortex-M3 Technical Reference Manual*. r2p1. ARM Limited. 2010.
- [Stm] *RM0008 Reference manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs*. Rev 21. STMicroelectronics. 2021.